IMSET
INSTITUT MAGHRÉBIN DES SCIENCES ÉCONOMIQUES ET DE TECHNOLOGIE
HONORIS UNITED UNIVERSITIES

ACADÉMIE D'ART DE CARTHAGE
HONORIS UNITED UNIVERSITIES

Intitulé du cours: Algorithmique et Programmation

Chapitre 3:

Les structures de contrôle répétitives
– La boucle "pour" –

Enseignante: Abid Arij

Année de formation 2024/2025

Objectifs du module

À la fin du module, l'apprenant doit être capable de :

- ✓ Comprendre et utiliser les structures conditionnelles dans la résolution algorithmique.
- ✓ Maîtriser les structures de contrôle répétitives utilisées en algorithmique, telles que les boucles **"pour" (for).**
  Ces structures sont essentielles pour automatiser l'exécution de blocs d'instructions selon une condition donnée ou un nombre déterminé de répétitions.

algorithme

Introduction

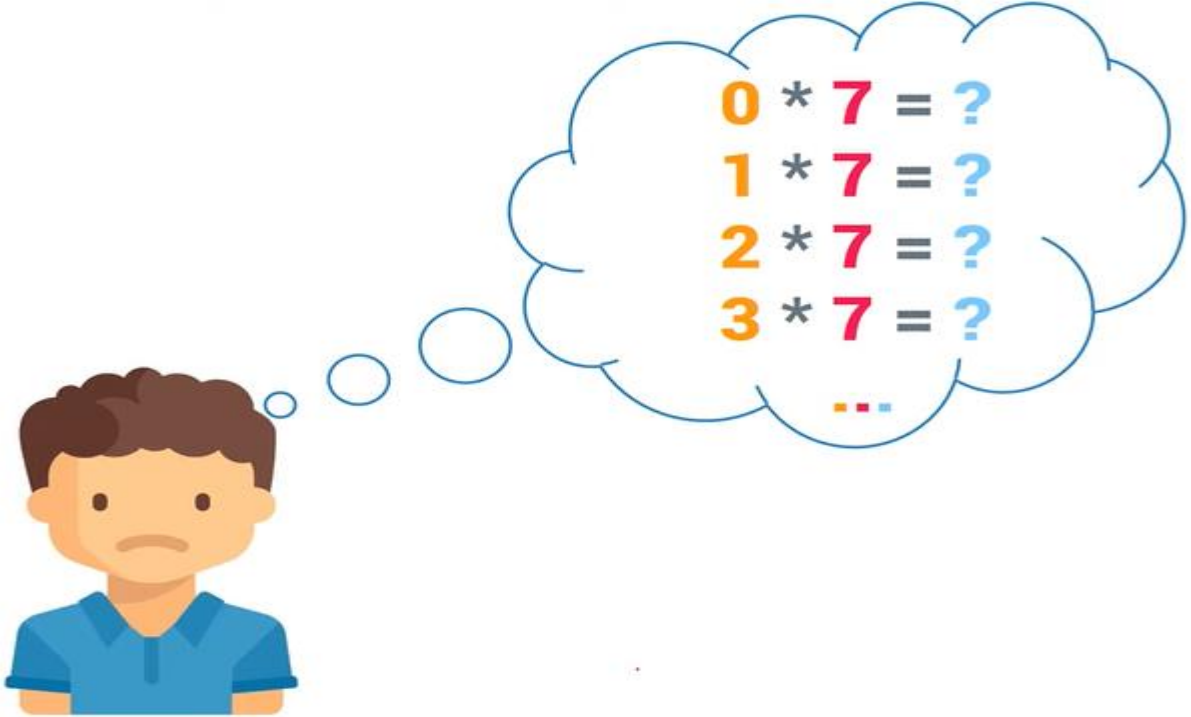


Table de multiplication de 7

0 * 7 = 0	6 * 7 = 42
1 * 7 = 7	7 * 7 = 49
2 * 7 = 14	8 * 7 = 56
3 * 7 = 21	9 * 7 = 63
4 * 7 = 28	10 * 7 = 70
5 * 7 = 35	

```
Algorithme table_7
Variables
        M : Entier
Début
        M ← 0 * 7
        Ecrire ( " 0 * 7 = " , M )
```

Table de multiplication de 7

```
Algorithme table_7
Variables
        M : Entier
Début
        M ← 0 * 7
        Ecrire ( " 0 * 7 = " , M )
        M ← 1 * 7
        Ecrire ( " 1 * 7 = " , M )
        M ← 2 * 7
        Ecrire ( " 2 * 7 = " , M )
        M ← 3 * 7
        Ecrire ( " 3 * 7 = " , M )
        M ← 4 * 7
        Ecrire ( " 4 * 7 = " , M )
        M ← 5 * 7
        Ecrire ( " 5 * 7 = " , M )
        M ← 6 * 7
        Ecrire ( " 6 * 7 = " , M )
        M ← 7 * 7
        Ecrire ( " 7 * 7 = " , M )
        M ← 8 * 7
        Ecrire ( " 8 * 7 = " , M )
        M ← 9 * 7
        Ecrire ( " 9 * 7 = " , M )
        M ← 10 * 7
        Ecrire ( " 10 * 7 = " , M )
Fin
```

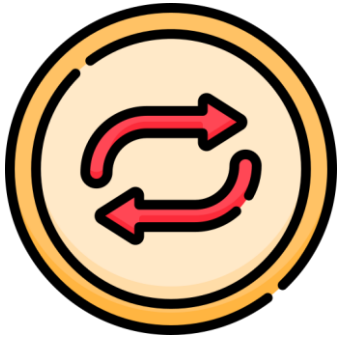
La structure « Pour … faire »

La **structure répétitive** (**Boucle**) permet d'exécuter plusieurs fois une séquence d'instructions.

Dans une boucle, le nombre de répétitions peut être connu, **fixé à l'avance**, comme il peut **dépendre d'une condition** permettant l'arrêt et la sortie de cette boucle.

Le nombre de répétitions peut être connu

Boucle Pour

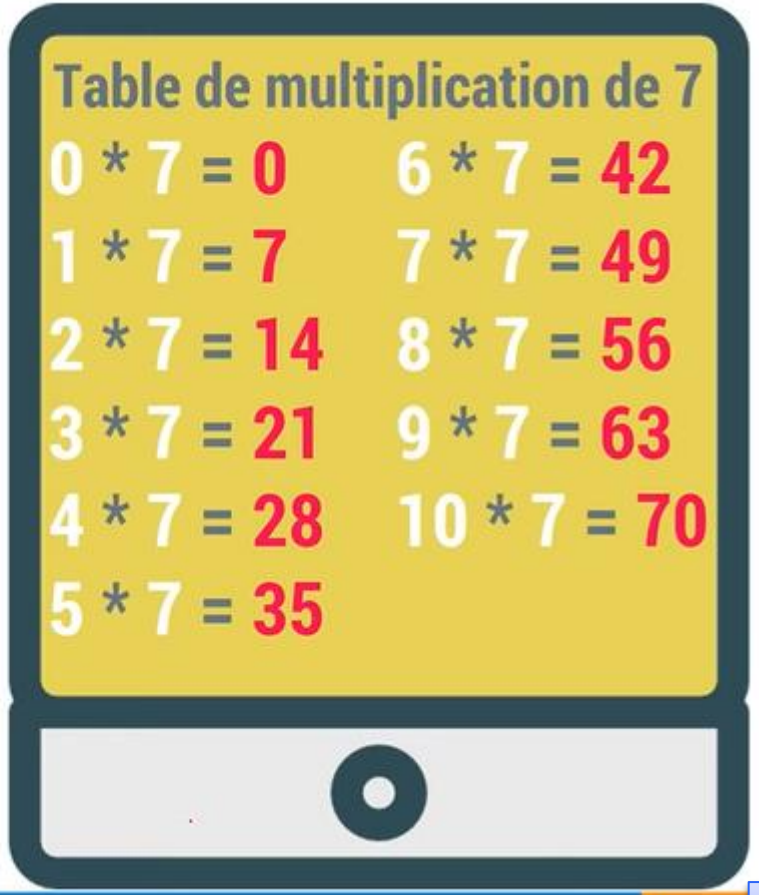


1 - La boucle Pour

Cette boucle permet d'exécuter une séquence d'instructions un nombre de fois connu fixé à l'avance. Elle utilise une variable (indice) de contrôle d'itérations caractérisée par :

- sa valeur initiale,
- sa valeur finale,
- son pas de variation.

```
Algorithme table_7
Variables
        M , i : Entier
Début
        Pour    i  de  0  à 10  faire
                M ← i * 7
                Ecrire ( i, " * 7 = " , M )
        Fin Pour
Fin
```



Algorithme : Table de multiplication de 7

```
Algorithme table_7
Variables
        M , i : Entier
Début
        Pour i de 0 à 10 pas 1 Faire
                M ← i * 7
                Ecrire ( i , " * 7 = " , M )
        Fin Pour
Fin
```

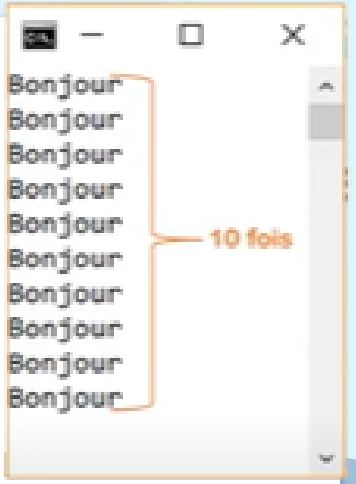
Solution avec une boucle

```
Algorithme table_7
Variables
        M : Entier
Début
        M ← 0 * 7
        Ecrire ( " 0 * 7 = " , M )
        M ← 1 * 7
        Ecrire ( " 1 * 7 = " , M )
        M ← 2 * 7
        Ecrire ( " 2 * 7 = " , M )
        M ← 3 * 7
        Ecrire ( " 3 * 7 = " , M )
        M ← 4 * 7
        Ecrire ( " 4 * 7 = " , M )
```

```
        M ← 5 * 7
        Ecrire ( " 5 * 7 = " , M )
        M ← 6 * 7
        Ecrire ( " 6 * 7 = " , M )
        M ← 7 * 7
        Ecrire ( " 7 * 7 = " , M )
        M ← 8 * 7
        Ecrire ( " 8 * 7 = " , M )
        M ← 9 * 7
        Ecrire ( " 9 * 7 = " , M )
        M ← 10 * 7
        Ecrire ( " 10 * 7 = " , M )
Fin
```

Solution sans utiliser les boucles

```
Ecrire (" Bonjour")
Ecrire (" Bonjour")
Ecrire (" Bonjour")
Ecrire (" Bonjour")
Ecrire (" Bonjour")
Ecrire (" Bonjour")
```




```
Algorithme affiche_bonjour
Var i: entier
Debut
Pour  i de 1  a 10 faire
      Ecrire (" Bonjour")

Fin pour
fin
```

Itération	Valeur de i	Action faite
1	4	Affiche "Bonjour"
2	5	Affiche "Bonjour"
3	6	Affiche "Bonjour"
4	7	Affiche "Bonjour"
5	8	Affiche "Bonjour"
6	9	Affiche "Bonjour"
7	10	Affiche "Bonjour"

Le programme affiche **7 fois** le mot **"Bonjour"** (une fois pour chaque valeur de i de 4 à 10).

•**Itération 1** : La boucle commence avec i = 4. Le programme écrit "Bonjour" une fois.
•**Itération 2** : i augmente de 1, donc maintenant i = 5. Le programme affiche à nouveau "Bonjour".
•**Itération 3** : i = 6, affiche "Bonjour".
•**Itération 4** : i = 7, affiche "Bonjour".
•**Itération 5** : i = 8, affiche "Bonjour".
•**Itération 6** : i = 9, affiche "Bonjour".
•**Itération 7** : i = 10, affiche "Bonjour".
Après cette 7e itération, i va augmenter à 11, qui est supérieur à la limite 10, donc la boucle s’arrête.

La structure « Pour … faire » (1)

1 - La boucle Pour

Syntaxe :

```
Pour I de N à M pas 1 Faire
    Instructions
Fin Pour
```

I : variable
N : valeur initiale
M : valeur finale
Le pas de variation égale à 1

La structure « Pour … faire » (1)

- La syntaxe de la structure **« *Pour … faire* »** est la suivante :

Pour *Compteur* ***de*** *ValDébut* ***à*** *ValFin* *[pas de ValPas]* ***faire***

action

Fin pour

- ***Compteur*** : Une variable qui sert à compter le nombre des itérations.
- ***ValDébut*** : C'est la valeur initiale du Compteur.
- ***ValFin*** : C'est la valeur finale du Compteur.
- ***ValPas*** : Une constante qui précise la valeur à ajouter ou à retrancher du Compteur lors de chaque itération. Par défaut, la valeur du **Pas** est égale à **1**.

Il ne faut jamais modifier la valeur du Compteur à l'intérieur de la boucle « Pour … faire »

La structure « Pour … faire » (2)

- **Remarques :**
  - On ne peut utiliser la boucle « Pour » que **si on connaît au préalable combien de fois** le traitement sera exécuté.
  - Le compteur de la boucle « Pour » **est incrémenté ou décrémenté automatiquement.**

Exercice Calculer la somme des entiers de 1 à N

- Écrire un algorithme qui lit un entier N et calcule la somme des entiers de 1 à N en utilisant une boucle **"pour"**.
- **Exemple avec N = 5 :**
  1 + 2 + 3 + 4 + 5 = 15

```
Début
  Lire (N)
  somme ← 0
  Pour i de 1 à N faire
    somme ← somme + i
  Fin Pour
  écrire (" La somme est : ", somme)
Fin
```

Itération (i)	Valeur de i	Calcul effectué	Nouvelle valeur de s
1	1	somme = 0 + 1	somme = 1
2	2	somme = 1 + 2	somme = 3
3	3	somme = 3 + 3	somme = 6
4	4	somme = 6 + 4	somme = 10
5	5	somme = 10 + 5	somme = 15

Explications pas à pas

- **Avant la boucle :**
  La variable **somme** est initialisée à 0, car on commence sans avoir rien ajouté.
- **Itération 1 (i = 1) :**
  On ajoute 1 à la somme actuelle (qui est 0).
  Donc somme devient 0 + 1 = 1.
- **Itération 2 (i = 2) :**
  On ajoute 2 à la somme actuelle (1).
  Donc somme devient 1 + 2 = 3.
- **Itération 3 (i = 3) :**
  On ajoute 3 à la somme actuelle (3).
  Donc somme devient 3 + 3 = 6.
- **Itération 4 (i = 4) :**
  On ajoute 4 à la somme actuelle (6).
  Donc somme devient 6 + 4 = 10.
- **Itération 5 (i = 5) :**
  On ajoute 5 à la somme actuelle (10).
  Donc somme devient 10 + 5 = 15.

Exercice : Affichage des nombres pairs entre 1 et 20

- Écrire un algorithme qui affiche tous les nombres pairs compris entre 1 et 20 inclus, en utilisant une boucle pour.

Correction détaillée :

Analyse :

- Un nombre est pair s'il est divisible par 2, donc si i mod 2 = 0.
- On utilise une boucle pour pour parcourir les nombres de 1 à 20.
- On teste si le nombre est pair avant de l'afficher.

```
Début
  Pour i  de 1 à 20 faire
    Si (i mod 2 = 0 )alors
      ecrire (i)
    Fin Si
  Fin Pour
Fin
```

Détail de toutes les itérations (chaque valeur de i et test de parité)

i	Calcul i % 2	i est pair ? (i % 2 = 0)	Affichage (Oui / Non)
1	1 % 2 = 1	Non	Non
2	2 % 2 = 0	Oui	Affiche 2
3	3 % 2 = 1	Non	Non
4	4 % 2 = 0	Oui	Affiche 4
5	5 % 2 = 1	Non	Non
6	6 % 2 = 0	Oui	Affiche 6
7	7 % 2 = 1	Non	Non
8	8 % 2 = 0	Oui	Affiche 8
9	9 % 2 = 1	Non	Non
10	10 % 2 = 0	Oui	Affiche 10
11	11 % 2 = 1	Non	Non
12	12 % 2 = 0	Oui	Affiche 12
13	13 % 2 = 1	Non	Non
14	14 % 2 = 0	Oui	Affiche 14
15	15 % 2 = 1	Non	Non

16	16 % 2 = 0	Oui	Affiche 16
17	17 % 2 = 1	Non	Non
18	18 % 2 = 0	Oui	Affiche 18
19	19 % 2 = 1	Non	Non
20	20 % 2 = 0	Oui	Affiche 20

- La boucle parcourt **20 itérations** (i = 1, 2, 3, ..., 20).
- À chaque itération, on calcule **i % 2** :
  - Si le résultat est 0 → **i est pair** → on affiche i.
  - Sinon, on ne fait rien (on passe à l'itération suivante).
- Les nombres affichés sont donc :
  2, 4, 6, 8, 10, 12, 14, 16, 18, 20

Écrire un algorithme nommé **somme** qui :

1. Demande à l'utilisateur de saisir un entier **positif** n.
2. Vérifie que n est bien un entier strictement positif.
   - Si ce n'est pas le cas, affiche un message d'erreur indiquant que l'entier saisi doit être positif.
3. Sinon, calcule la somme des entiers de 1 jusqu'à n.
4. Affiche ensuite cette somme.

i	Calcul effectué	Valeur de s après l'itération
1	s = 0 + 1	1
2	s = 1 + 2	3
3	s = 3 + 3	6
4	s = 6 + 4	10
5	s = 10 + 5	15

```
Algorithme Somme
Variables
  n, i, s : Entier
Début
  Écrire("Saisir un entier positif : ")
  Lire(n)

  Si (n <= 0 ) alors
    Écrire("L'entier saisi doit être positif.")
  Sinon
    s ← 0
    Pour i de 1 à n faire
      s ← s + i
    Fin Pour
    Écrire("La somme est : ", s)
  Fin Si
Fin
```

Exercice :

- Écrire un algorithme qui calcule la somme des éléments de la suite suivante :
  2 + 5 + 8 + 11 + … + 101

```
Algorithme SommeSuitePour
Var
   i, somme : entier
Début
   somme ← 0
   Pour i de 2 à 101 par pas de 3 faire
      somme ← somme + i
   FinPour
   Ecrire("Somme = ", somme)
Fin
```

Écrire un algorithme qui lit un entier **positif N** puis affiche **tous ses diviseurs**.

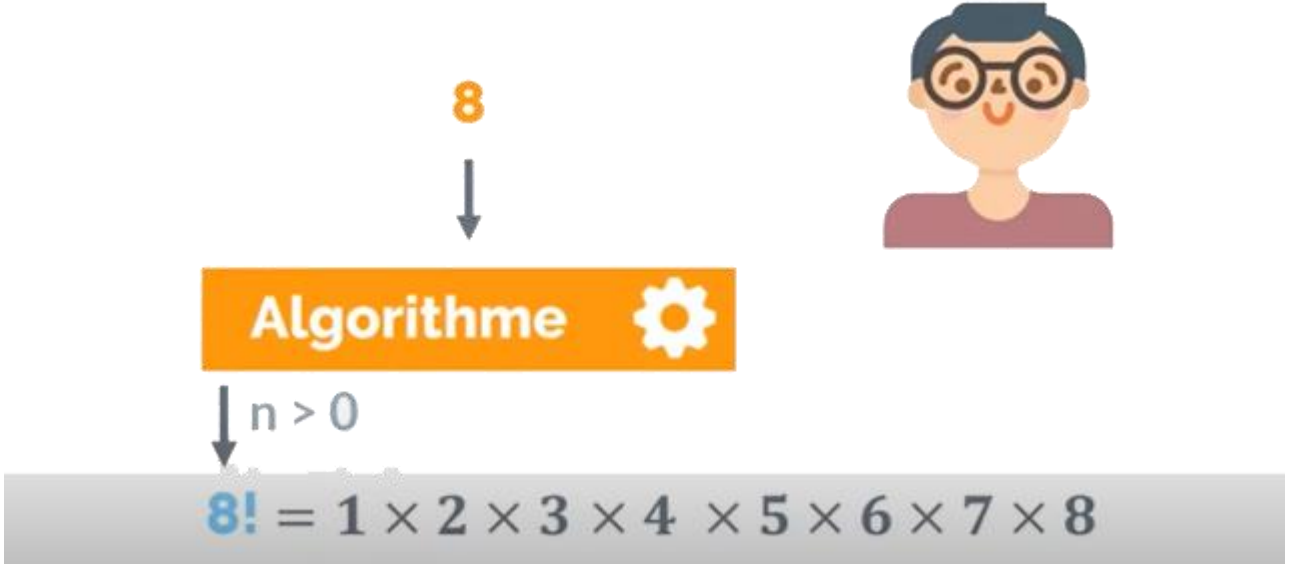
- **Diviseurs =>** (les nombres qui divisent N sans reste).

Exemple réel : N = 12
On cherche les diviseurs de 12.
•i = 1 → 12 mod 1 = 0 → afficher 1
•i = 2 → 12 mod 2 = 0 → afficher 2
•i = 3 → 12 mod 3 = 0 → afficher 3
•i = 4 → 12 mod 4 = 0 → afficher 4
•i = 5 → 12 mod 5 = 2
•i = 6 → 12 mod 6 = 0 → afficher 6
•i = 7 → 12 mod 7 = 5
•i = 8 → 12 mod 8 = 4
•i = 9 → 12 mod 9 = 3
•i = 10 → 12 mod 10 = 2
•i = 11 → 12 mod 11 = 1
•i = 12 → 12 mod 12 = 0 → afficher 12

Résultat final : 1, 2, 3, 4, 6, 12
Nombre total d'itérations : 12

```
Algorithme Diviseurs
Var
   N, i : entier
Début
   Ecrire("Donner N :") ;
    Lire(N)
   Pour i de 1 à N faire
      Si (N MOD i = 0 ) alors
         Ecrire(i)
      FinSi
   FinPour
Fin
```

- La **factorielle** d'un nombre **n** (notée n!) est le **produit de tous les entiers de 1 à n** :
- Exemple :
  5! = 1 × 2 × 3 × 4 × 5 = 120
  3! = 1 × 2 × 3 = 6
  1! = 1
  0! = 1 (par convention mathématique)



```
Algorithme factorielle
Variables
   n, i, f : Entier
Début
   Écrire("Saisir un entier  ")
   Lire(n)
    fact ← 1
Pour i de 1 à n faire
        fact ← fact × i
    Fin Pour
 Écrire(" La factorielle d'un nombre n est : ", fact)
Fin
```

i	fact (avant)	Calcul	fact (après)
1	1	1 × 1	1
2	1	1 × 2	2
3	2	2 × 3	6
4	6	6 × 4	24

Exercice :Écrire un algorithme qui affiche le carré de chaque entier entre 1 et 4 (1^2, 2^2, ..., 4^2).

```
Algorithme Affiche_Carres
Var i, carré : entier
Début
   Pour i de 1 à 4 faire
      carré ← i * i
      Ecrire("Le carré de ", i, " est ", carré)
   FinPour
Fin
```

Étape	Valeur de i	Calcul	Affichage
1	i = 1	1 * 1 = 1	Le carré de 1 est 1
2	i = 2	2 * 2 = 4	Le carré de 2 est 4
3	i = 3	3 * 3 = 9	Le carré de 3 est 9
4	i = 4	4 * 4 = 16	Le carré de 4 est 16
5	i = 5 > 4	fin de boucle	

Exercice : Écrire un algorithme qui compte le nombre de nombres impairs entre 1 et 30 et affiche le résultat.

```
Algorithme compteNombre
Var i, compteur : entier
Début
   compteur ← 0
   Pour i de 1 à 30 faire
      Si (i mod 2 <> 0 ) alors
         compteur ← compteur + 1
      Fin Si
   Fin Pour
   ecrire ("Nombre des entiers impairs :", compteur)

Fin
```

i	i mod 2	Impair ?	compteur
1	1	Oui (1 ≠ 0)	1
2	0	Non	1
3	1	Oui	2
4	0	Non	2
5	1	Oui	3
6	0	Non	3
7	1	Oui	4
8	0	Non	4

Exercice : Afficher tous les nombres entre **10 et 99** dont la **somme des chiffres** **est égale à 9.**

```
Algorithme sommeChiffres
Var i, somme, unité, dizaine : entier
Début
  Pour i de 10 à 99 faire
     unité ← n modulo 10
     dizaine ← n div 10
     somme ← unité + dizaine
     Si (somme = 9) alors
        Écrire (i)
     Fin Si
  Fin Pour
Fin
```

Exemple 1 :

n = 18

unité = 18 mod 10 = 8

dizaine = 18 div 10 = 1

somme = 8 + 1 = **9** ✅

On affiche **18**

Exemple 2 :

n = 45

unité = 45 mod 10 = 5

dizaine = 45 div 10 = 4

somme = 5 + 4 = 9 ✅

On affiche **45**

Merci pour votre attention